

Customer Churn and Customer Spending Prediction in Telecommunications Using Machine Learning

A supervised learning analysis of customer churn drivers and monthly spending behavior using Telco data

ITC6103B1 - Applied Machine Learning – Winter Term 2026

Kimon Lappas
Ioannis Logothetis
Milena Mirumyan
Katerina Psallida



Table of Contents:

Introduction	4
The Business Problem	4
Data Preprocessing & Exploratory Data Analysis	5
High Cardinality Columns.....	5
Multiple Categorical Features	5
Total Charges Feature Handling.....	6
Feature Engineering	7
Feature Selection	7
Finalizing the Regression Target	9
A Speculation from Exploratory Data Analysis	10
Supervised Classification Analysis	12
Dataset Structure and Class Distribution.....	13
Additional exploration relevant to classification Models	14
Data Preparation – Train/Test split.....	14
Model Development and Results	15
Random Forest (RF) – Base model	15
Random Forest (RF) – Tuned model.....	15
Random Forest (RF) – Model Comparison	16
SVM – Base model.....	17
SVM – Tuned model	17
SVM – Model Comparison	18
Model Performance Comparison	18
Recommended Model: SVM Tuned	18
Model Interpretation	19
Regression Analysis.....	20
One Hot Encoder Transformation with ColumnTransformer	20
As mentioned in the analysis “Multiple Categorical Features” in EDA, the “.get_dummies” was applied to the whole dataset for encoding the categorical variables.	20
Correlation between variables after OHE	21
Baseline Regression Models.....	23

Feature Engineering	24
Model Interpretation	26
Most important parameters affecting customer spend	30
Additional Model Approach.....	30
Churn Classification	30
Tuning & Best Parameters	30
Results.....	31
What Is Driving Churn? (SHAP)	32
Monthly Charge Regression Analysis	33
Tuning & Best Parameters	33
Results.....	33
Actual vs Predicted and Residuals.....	34
What Drives Monthly Charges? (SHAP)	35
Overall Conclusions	36

Introduction

We gratefully acknowledge IBM for generating and sharing the dataset. The data can be accessed via [Kaggle](#) at the following link: [Telco Customer Churn Dataset](#)

The dataset is well suited for the scope of the Applied Machine Learning (from now on, ML) project, as it presents an intermediate-to-easy level of preprocessing complexity and includes both classification and regression targets. Specifically, it contains a binary churn label for classification tasks and continuous financial variables (e.g., charges) suitable for regression analysis. This allows the project to focus primarily on applied machine learning concepts rather than extensive data validation, interpretation, or complex preprocessing.

Although the Kaggle page primarily provides access to the raw dataset in CSV format, additional contextual information about the business problem is available through IBM's official community resources. The page is available here at [IBM's Page](#).

For feature explanation, the ["Appendix A – Feature Explanation"](#) may be visited for an explanation of the features of the dataset.

The Business Problem

For convenience, it is assumed that IBM is the telecommunications company represented in the data. It is also assumed that IBM cares about interpretability.

The Telco Customer Churn dataset represents a snapshot of customer behavior at a specific point in time, capturing demographic, service, and billing information. As mentioned before, the dataset supports both classification and regression problems. Therefore, the business scope of this project is to build two ML models: one for classification, to investigate and predict what factors lead a customer to churn, and one for regression, to investigate and predict what factors affect pricing strategies.

To ensure that the best models are implemented into production, more than two candidate models need to be explored, so that the best-performing one is selected for each task.

If stakeholders try to summarize the problem using questions, the most relevant business questions addressed by this project are:

Did this customer leave during the most recent quarter?

Do contract policies and pricing strategies require improvement?

Concerning the business impact, the classification of ML model will help the telco company retain customers and maintain revenue and market share. The regression model will help IBM understand customer spending patterns. A regression analysis on charges, and specifically on the feature-engineered average monthly charges, can help identify factors that drive high or low billing amounts. Therefore, the regression analysis further assists the business in identifying the most profitable contract features, detecting customers with unusually low or high spending, and informing pricing strategies.

Assuming that the best two models have been selected, the final real-life step would be to deploy them into production. This means that the ML pipeline continuously reads new data and generates predictions on whether a customer will churn or not. These predictions are then compared with actual outcomes (did the customer churn?) to monitor model performance. If performance drops, investigation by ML engineers is required so that the models can be retrained and re-tuned, and potential new factors can be re-evaluated with stakeholders to support strategic decisions for reducing churn.

Regarding regression, a performance drop may indicate a mismatch between the factors stakeholders consider important for pricing and the real-world drivers. In this case, re-optimization is needed both for the regression model and for real-life business actions.

Data Preprocessing & Exploratory Data Analysis

High Cardinality Columns

There is only one feature which is of high cardinality, and this is the unique identifier “customerID”, and is dropped since it does not add any information that can be analyzed.

Binary Categorical Features

All features are processed using the same binary categorical handling technique: their values are transformed to True/False using the `.map()` method. This approach is chosen because True/False is equivalent to 1/0 for ML models, while maximizing interpretability.

Multiple Categorical Features

Two approaches are followed: A standard one, with `OneHotEncoder` instance fitted only on train set and transforming both train and test set and a second one, not so standard, in which `Pandas .get_dummies()` is applied before train and test splits.

The results of both approaches are compared with each other demonstrating that applying `get_dummies()` before splitting does not affect the model in this situation.

Applying `get_dummies()` before the train-test split facilitates the application of predictions later, since it guarantees that the columns of the train and test sets will be the same. Additionally, this enhances interpretability during EDA, since all categories are represented as True/False values and allow for consistent visualization.

On the other hand, if the set of columns changes, the process may break. Additionally, the fact that the columns for train and test are the same means that the training set is aware of all categories appearing in the test set. Methodologically speaking, this implies that future information about categories is available during training. In the case of this dataset, where even the rarest categorical values appear in hundreds of observations, the probability that all instances of a category appear only in the test set (and are therefore unseen during training) is extremely low.

Total Charges Feature Handling

This feature is a preprocessing trap because it contains eleven empty values that Pandas does not recognize as valid nulls. As a first step, all empty values are converted to valid nulls. Upon inspection, it is observed that all these null values correspond to customers in their first month of the contract. Therefore, the appropriate treatment is to fill these nulls with zero, representing 0 euros.

Filling missing values with `fillna()` before the train–test split is safe because, unlike statistical imputation techniques, it does not compute any values from the dataset and therefore does not introduce data leakage.

Outliers handling

To handle outliers, the numerical features are considered separately (Tenure, Monthly_Charges, Total_Charges). A figure with three subplots can then be created to display the boxplots for these features. As shown in **Error! Reference source not found.**, there are no apparent outliers in any of these features.

**Dataset does not contain any outliers.
Units: Tenure in months, Charges in €**

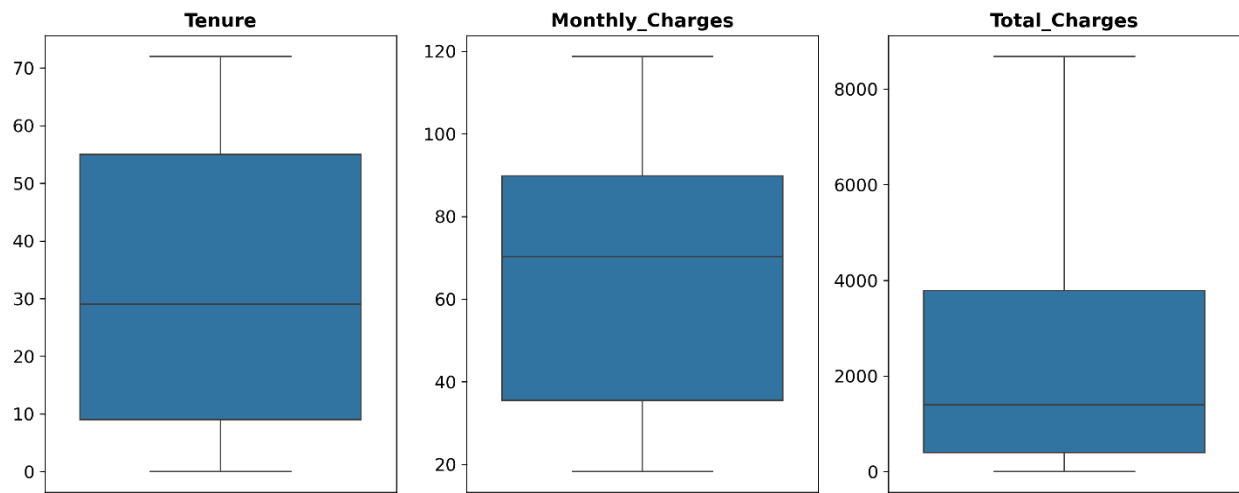


Figure 1 Boxplots of numerical features demonstrate that the dataset does not contain any outlier.

Feature Engineering

A new continuous feature to normalize total charges is created by applying the formula:

$$Avg_Monthly_Charge = \frac{Total_Charges}{Tenure}$$

The Avg_Monthly_Charge feature was created to normalize customer spending by tenure. Its low correlation with tenure (0.25) allows fair comparison between short- and long-term customers, while its moderate correlation with total charges (0.65) ensures it still captures overall customer value. This makes it more informative for ML prediction than Total_Charges alone. For this reason, Avg_Monthly_Charge is preferred over Total_Charges for regression tasks.

Row-wise transformations such the one is used here are safe and do not compromise the integrity of the train-test split.

Feature Selection

Since correlation analysis was already performed above, the same method is used for feature selection. **Error! Reference source not found.** demonstrates why the Monthly_Charges feature can be safely removed from the dataset:

Correlation Table	Tenure	Monthly_Charge	Total_Charges
Avg_Monthly_Charge	0.249	0.994	0.651

Table 1 Correlation table of average monthly charges and remaining numerical features.

Such a strong correlation indicates that both features carry almost identical information, which can lead to redundancy and multicollinearity in the model.

Categorical Features Frequencies

Error! Reference source not found. presents the distribution of some of the dataset's binary features using a figure instance, with subplots and Seaborn's countplot. The distributions appear reasonable, which is a good indication of a valid dataset. Additionally, the number of rows for all categorical values is sufficient to support effective ML model training.

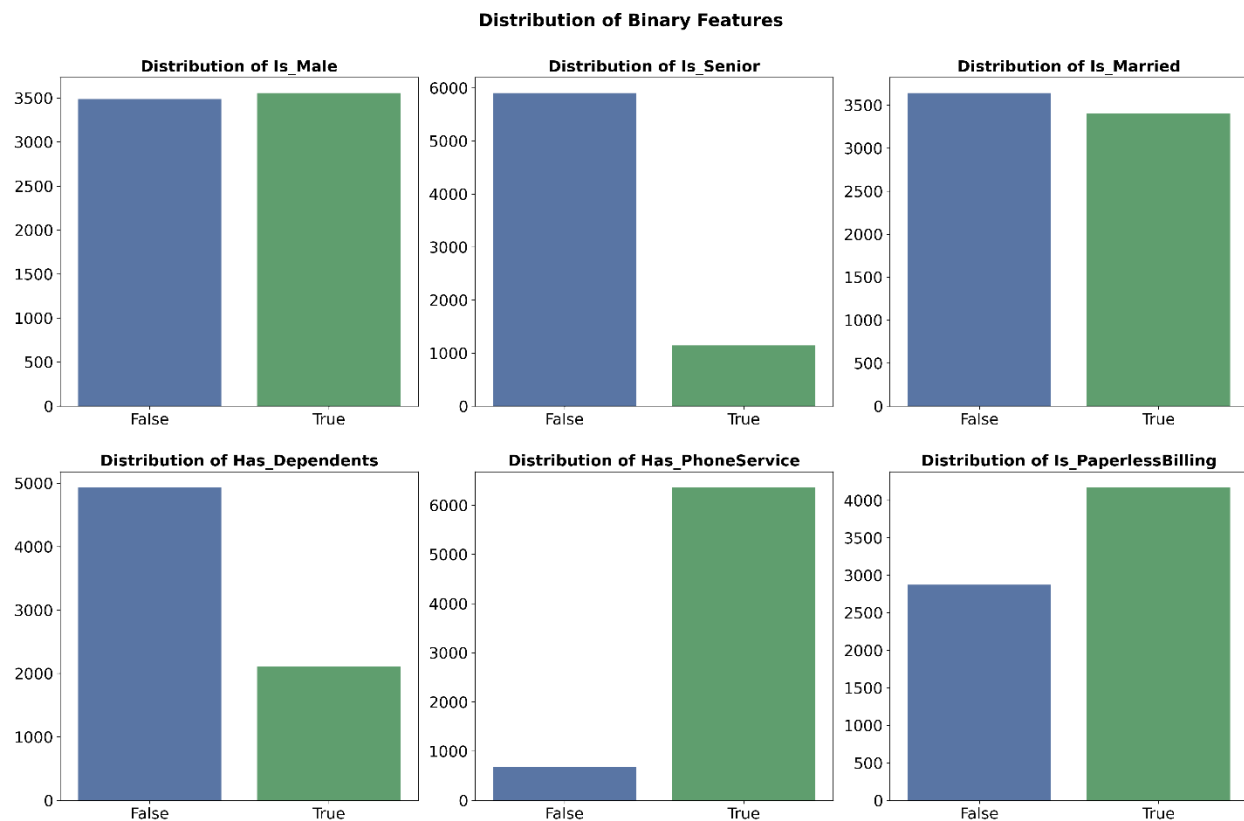


Figure 2 Distributions of binary features present the distribution of Row Counts of all One-Hot Encoded Features.

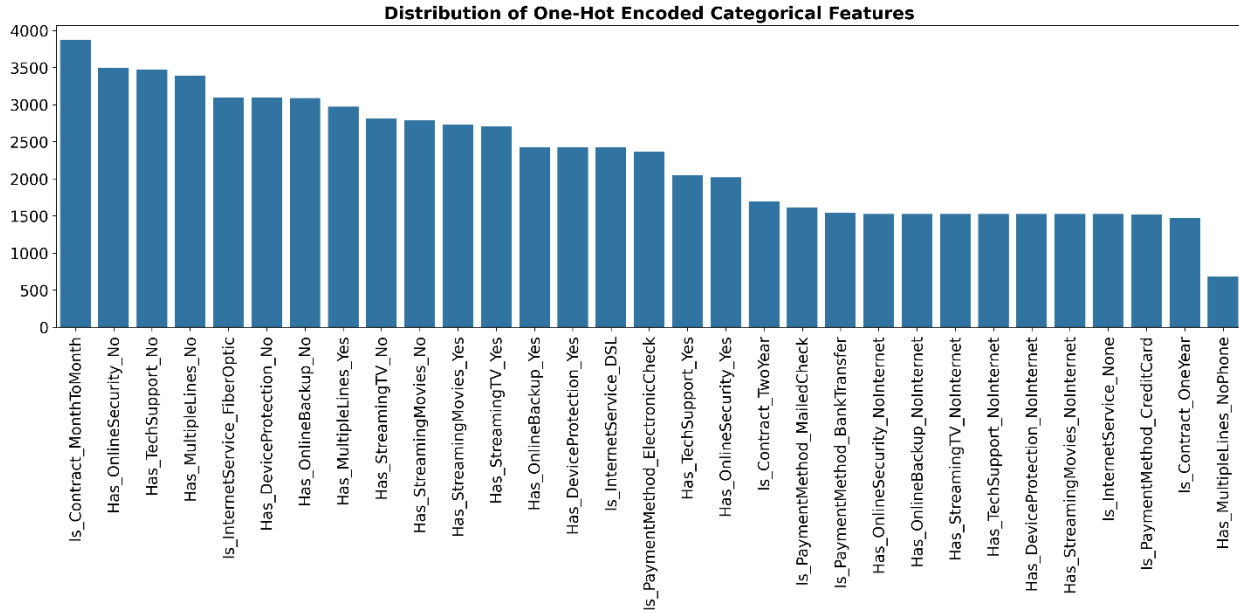


Figure 3 Distribution of row counts for all One-Hot Encoded features

The visualizations confirm that there is no need to group rare categories or drop entire columns. Understanding category frequencies ensures that all features are sufficiently represented and helps determine whether additional preprocessing, such as rebalancing or encoding adjustments, is required.

Finalizing the Regression Target

Correlation analysis of all remaining numerical features is useful for identifying the final regression target. The high correlation (0.83) between Total_Charges and Tenure indicates that Total_Charges is strongly influenced by how long a customer has been active. In contrast, Avg_Monthly_Charge has a much lower correlation with Tenure (0.25), making it a more unbiased measure of customer spending. The correlation analysis of the final numerical features can be seen in **Error! Reference source not found..**

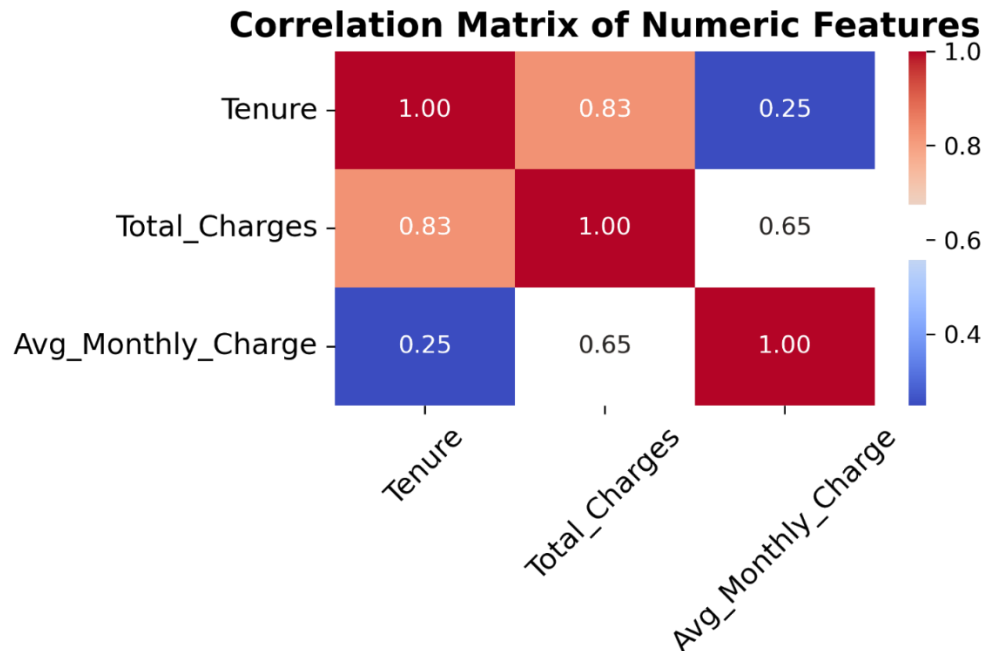


Figure 4 Correlation heatmap demonstrating that average monthly charges is the best option for regression target.

Error! Reference source not found.4 shows that customers with short-tenure contracts and higher charges are more likely to churn. This pattern is reasonable and aligns with expectations from ML models.

A Speculation from Exploratory Data Analysis

Figure 5 shows that customers with short-tenure contracts and higher charges are more likely to churn. This pattern is reasonable and aligns with expectations from ML models.

Scatterplot of Avg Monthly Charge vs Tenure

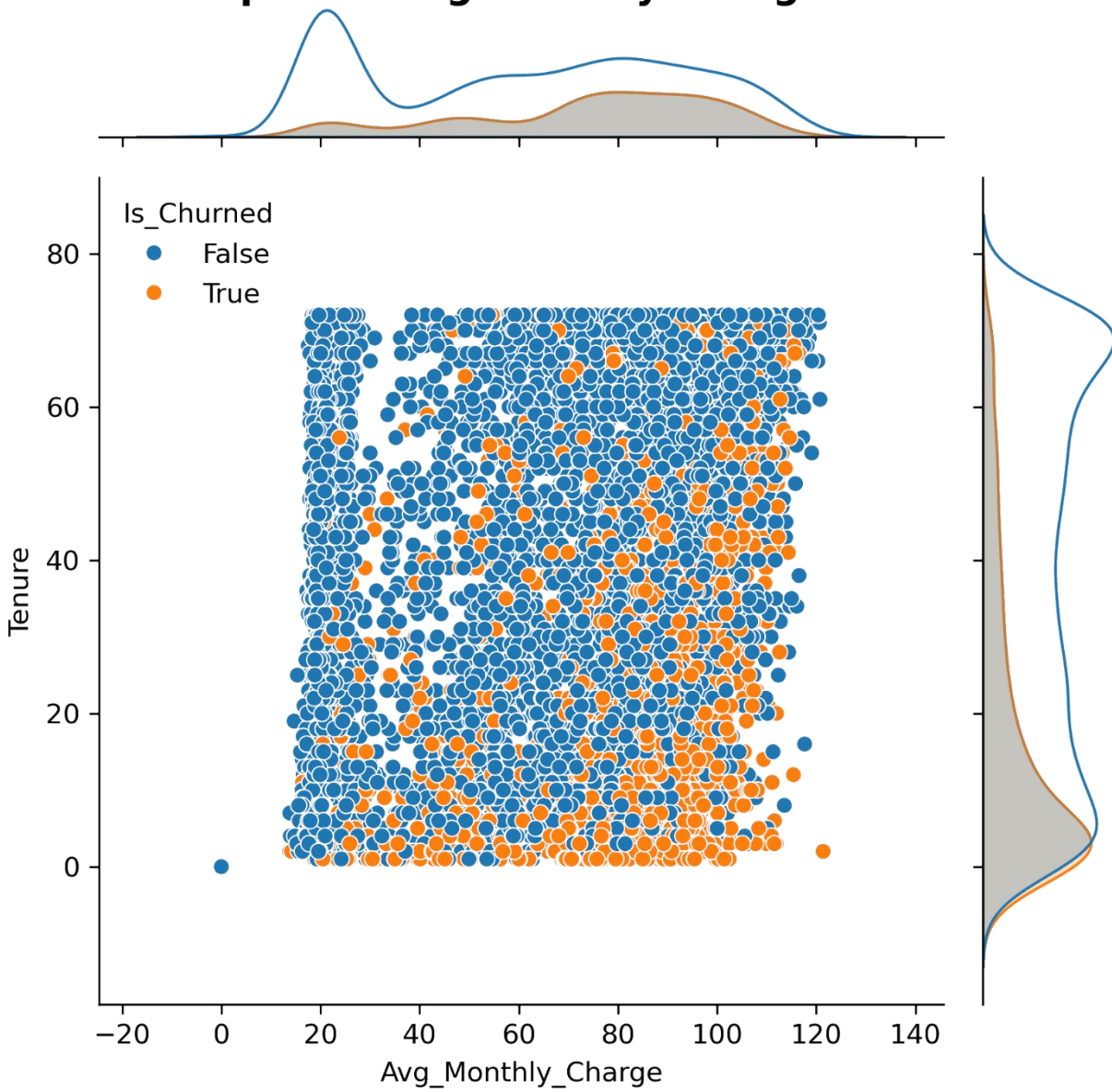


Figure 5 High average monthly costs, combined with low tenure, are highly likely to lead to churn.

Customer ID: A unique identifier

Gender: The customer's gender (Male, Female)

Senior Citizen: Indicates if the customer is 65 or older (1: True, 0: False)

Married: Indicates if the customer is married (Yes, No)

Dependents: Indicates if the customer lives with any dependents, i.e. children, parents, etc. (Yes, No)

Tenure in Months: Indicates the total amount of months that the customer has been with the company until this quarter

Phone Service: Indicates if the customer subscribes to home phone service with the company (Yes, No)

Multiple Lines: Indicates if the customer subscribes to multiple telephone lines with the company (Yes, No)

Internet Service: Indicates if the customer subscribes to Internet service with the company (No, DSL, Fiber Optic)

Online Security: Indicates if the customer subscribes to an additional online security service provided by the company (Yes, No)

Online Backup: Indicates if the customer subscribes to an additional online backup service provided by the company (Yes, No)

Device Protection Plan: Indicates if the customer subscribes to an additional device protection plan for their Internet equipment provided by the company (Yes, No)

Premium Tech Support: Indicates if the customer subscribes to an additional technical support plan from the company with reduced wait times (Yes, No)

Streaming TV: Indicates if the customer uses their Internet service to stream television programming from a third-party provider (Yes, No)

Streaming Movies: Indicates if the customer uses their Internet service to stream movies from a third-party provider (Yes, No)

Contract: Indicates the customer's current contract type (Month-to-Month, One Year, Two Year)

Paperless Billing: Indicates if the customer has chosen paperless billing (Yes, No)

Payment Method: Indicates how the customer pays their bill (Bank Withdrawal - Automatic, Credit Card - Automatic, Mailed Check, Electronic Check)

Monthly Charge: Indicates the customer's current total monthly charge for all their services from the company

Total Charges: Indicates the customer's total charges, calculated until this quarter

Churn Label: Yes = the customer left the company this quarter. No = the customer remained with the company. Directly related to Churn Value.

Table 2 Feature Explanation

Supervised Classification Analysis

This chapter explores the supervised classification models of the Telco churn project, building directly from the cleaned dataset that was produced in prior preprocessing phase. The chapter explored two different models of Random Forest and two of SVM with RBF kernel. One model of each was a baseline one, while the other two were tuned and optimized by RandomizedSearchCV.

Model qualities were evaluated by Accuracy, Precision, Recall, F1-Score and ROC-AUC. The best classification model turned out to be the tuned SVM. Because of the imbalanced nature of churn rate settings, accuracy alone is not sufficient to predict how correct the models are, so the final recommendation of the best model combines all the above metrics for the final decision of the best model.

The added ROC-AUC is an important metric that identifies how well the model can distinguish classes. In cases such as churn rate, it adds additional value because it provides a single number between 0-1, making it useful in imbalanced problems, where relying only on accuracy and f1-scores may not give accurately the whole picture of how well the model separates churners from non-churners.

Dataset Structure and Class Distribution

Attribute	Value
Total records	7,043
Total columns	31
Predictor variables	30
Numeric features	3 (Tenure, Total_Charges, Avg_Monthly_Charge)
Boolean features	27 (encoded categorical variables)
Missing values	0
Non-churned (False)	5,174 (73.46%)
Churned (True)	1,869 (26.54%)

Table 3 Dataset summary statistics extracted from the ML-ready Parquet file.

As observed the dataset exhibits a Total of 7,043 records with around 27% churned and 73% non-churned. This sets the minimum performance level that we expect every useful model to

reach. Although a basic metric, it is a core benchmark in identifying actual churners, if the models predict worse than reality, they are not valuable for actual decision making.

Additional exploration relevant to classification Models

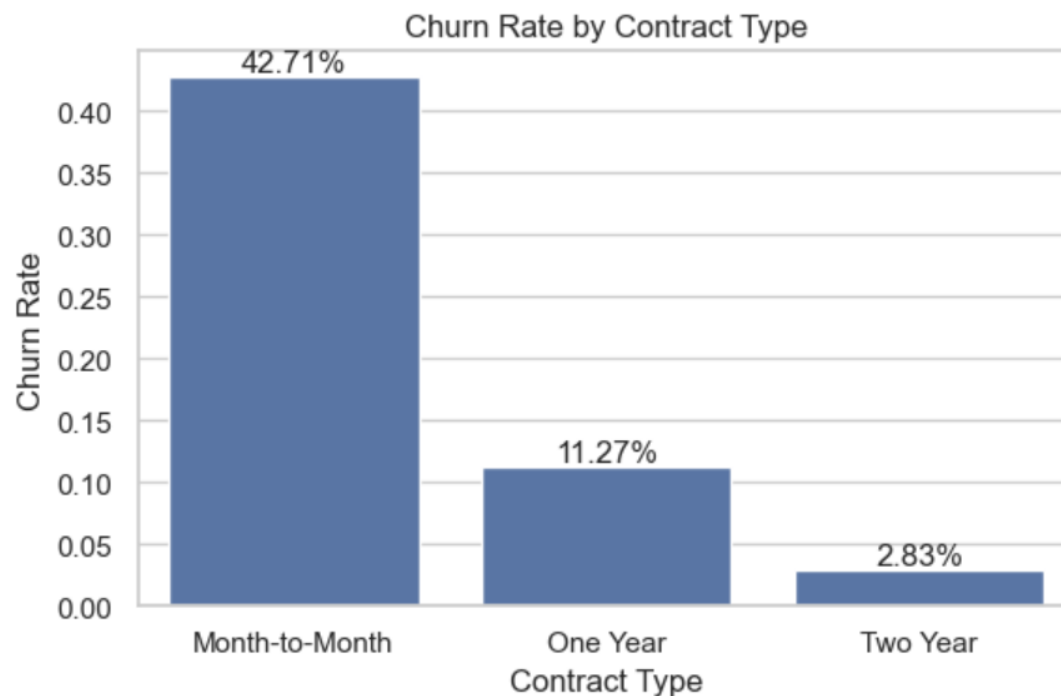


Figure 6 Churn rate by contract type. Month-to-month customers churn at 15x the rate of two-year contract holders.

Additionally, contract type was visualized to identify the most extreme outlier in dataset. The gap between MtM, One- and Two-year customers is more than 30% and 39% accordingly. This will be further explored in feature importance, where contract-related variables consistently rank among the top predictors.

Data Preparation – Train/Test split

To create an accurate model, all Boolean predictor columns were converted to integers to produce a fully numeric matrix. Then a stratified 70/30 split was applied to preserve the according class ration in both subsets.

Subset	Records	Non-Churned	Churned
Training (70%)	4,930	73.47%	26.53%
Test (30%)	2,113	73.45%	26.55%

Table 4 Train / Test stats

Stratification is extremely important because it ensures that the churn proportions of train and test sets remain constant with the whole dataset at around 27%, thus preventing evaluation of distortion.

Model Development and Results

Random Forest (RF) – Base model

The default RF produced a train/test accuracy of 0.9980/ 0.7894, indicating strong overfitting. The model clearly memorized training data rather than generalizing. Churn recall of 0.4848 means that fewer than half of actual churners were identified, which is insufficient for retention use. Finally, a ROC-AUC of 0.8216 confirms good discriminative capacity, but the classification threshold suppresses recall.

	Model	Train Accuracy	Test Accuracy	Precision	Recall	F1-score	ROC-AUC
0	Random Forest Baseline	0.998	0.7894	0.6355	0.4848	0.5501	0.8216

Figure Baseline RF Model Performance Summary

Based on the above results, a better model would be needed to properly satisfy business needs.

Random Forest (RF) – Tuned model

The more optimized RF model focuses on optimization using RandomizedSearchCV (scoring = F1, CV best score = 0.5777), with the objective of improving generalization and better balance

between precision and recall. The following table presents the selected hyperparameters

Parameter	Value	Purpose
n_estimators	100	Stable ensemble with manageable compute
max_depth	15	Limits tree complexity to prevent overfitting
min_samples_leaf	6	Reduces noise at terminal nodes
min_samples_split	2	Allows flexible internal splits
max_features	log2	Controls feature randomness per split
bootstrap	True	Enables bagging diversity across trees

Table 5 Random Forest Hyperparameters

Random Forest (RF) – Model Comparison

	Model	Train Accuracy	Test Accuracy	Precision	Recall	F1-score	ROC-AUC
0	Random Forest Baseline	0.9980	0.7894	0.6355	0.4848	0.5501	0.8216
1	Random Forest Tuned	0.8444	0.7984	0.6675	0.4795	0.5581	0.8416

Table 6 RF Model Performance Comparison

As noticed from the results, Tuning resolves overfitting by a huge margin. The train/test gap narrows from 21 pp to just 4.6 pp. Test accuracy improves to 0.7984, precision rises to 0.6675, and ROC-AUC reaches 0.8416.

However, recall remains low at 0.4795, which indicates that tuning improved stability and precision but did not change the model's general conservative prediction rates.

SVM – Base model

	Model	Train Accuracy	Test Accuracy	Precision	Recall	F1-score	ROC-AUC
0	SVM RBF Baseline	0.7673	0.7402	0.5069	0.7807	0.6147	0.8199

Table 7 Baseline SVM (RBF Kernel) Model Performance Summary

The baseline SVM uses StandardScaler to normalize feature values accordingly and avoid leakage. With the RBF kernel's assistance, it captures nonlinearity in the data, while `class_weight='balanced'` compensates for class imbalance to give weight to the churned class during training. This step was critical since the shift of the decision boundary produced a recall of 0.7807, which is more than 60% higher than RF's 0.4848. Additionally, compared to RF, precision dropped to 0.5059 with more false positives being identified. Finally, a train/test accuracy of 0.7673/0.7402 signals no actual overfitting.

SVM – Tuned model

The tuned SVM model tries to improve the baseline through RandomizedSearchCV with F1-score as the primary optimization metric.

Parameter	Value	Effect
C	0.1	Low penalty — prioritises wider margin, less overfitting
gamma	0.1	Moderate locality — balanced decision boundary
kernel	RBF	Nonlinear separation in high dimensional space
class_weight	balanced	Compensates for churn class imbalance

Table 8 Tuned SVM (RBF Kernel) Hyperparameter Configurations

SVM – Model Comparison

	Model	Train Accuracy	Test Accuracy	Precision	Recall	F1-score	ROC-AUC
0	SVM RBF Baseline	0.7673	0.7402	0.5069	0.7807	0.6147	0.8199
1	SVM RBF Tuned	0.7753	0.7440	0.5122	0.7504	0.6088	0.8176

Table 9 SVM Model Performance Comparison

As observed from the comparison table above, tuning produced marginal changes. Accuracy and precision were slightly improved, while Recall and F1-score declined. This is because the base model was already operating near its optimum configuration.

Model Performance Comparison

Model	Train Acc.	Test Acc.	Precision	Recall	F1-Score	ROC-AUC
RF Baseline	0.9980	0.7894	0.6355	0.4848	0.5501	0.8216
RF Tuned	0.8444	0.7984	0.6675	0.4795	0.5581	0.8416
SVM Baseline	0.7673	0.7402	0.5069	0.7807	0.6147	0.8199
SVM Tuned	0.7753	0.7440	0.5122	0.7504	0.6088	0.8176

Table 10 Final Model Performance Comparison (Random Forest vs SVM)

The results highlight clear differences between RF and SVM models. RF models achieve higher accuracy, precision, and ROC-AUC, especially at the tuned model, while SVM shows clear advantages in recall and F1-score, indicating stronger performance in detecting churners.

Recommended Model: SVM Tuned

The recommended model is the tuned SVM RBF. While Random Forest achieves higher accuracy (0.7984 vs 0.7440) and ROC-AUC (0.8416 vs 0.8176), these advantages do not translate to better business value for this task.

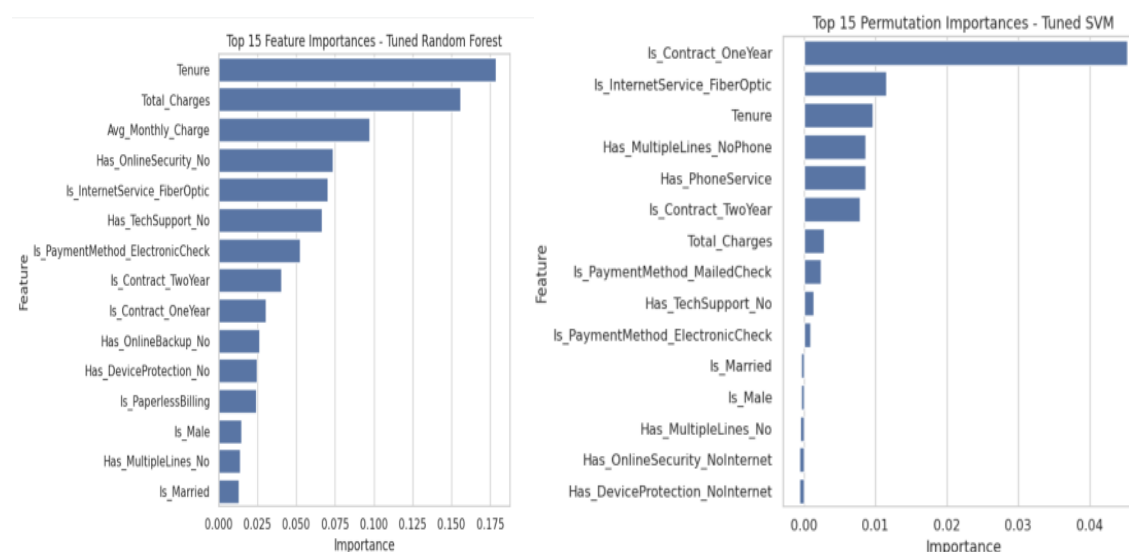
The SVM detects 75% of true churners vs 48% for Random Forest, a difference of 27% in recall. In practical terms, over a test set of 561 churned customers, the SVM correctly identifies approximately 421 versus 269 for Random Forest. This represents 152 additional at-risk customers captured per cycle, each potentially retainable through targeted intervention.

Model Interpretation

Since models such as Random Forest and especially SVM are considered black-box models, their decision-making process is not easily interpretable, which creates challenges for business use where understanding why a customer is predicted to churn is essential for targeted actions and strategy design. To address this, we explored explainability techniques to identify the key drivers of churn predictions.

Initially, SHAP was applied to both tuned models to provide a unified and detailed interpretation of feature contributions. However, in practice, SHAP proved to be computationally expensive, particularly for the SVM model (43% in more than an hour), where it relies on slower model-agnostic approximations. This resulted in very long execution times, making it impractical for this project.

As a result, we adopted alternative methods better suited to each model. For Random Forest, we used its built-in feature importance, which identified tenure, total charges, and monthly charges as the most influential predictors. For SVM, we applied permutation importance, which showed that contract type and internet service are key drivers of churn. Although less granular than SHAP, permutation importance provides a reliable and computationally efficient way to interpret model behavior, while still delivering actionable insights for business decision-making.



Regression Analysis

Assumptions during this analysis were the following:

- The variable to be forecasted was selected to be the Avg_Monthly_Charges. This is the engineered feature which accounts for the total dollars that somebody is paying, regardless of the contract. Under this assumption, the prediction to be made is not which contract is to be sold to a customer but rather what the **customer spend will be operational**.
- Monthly_Charges were dropped, as the field was highly correlated with the Avg_Monthly_Charges (as seen in EDA - Table 1)
- Total_Charges were dropped, as our predicted variable is derived from it directly with a multiplication factor (the Tenure)
- Is_Churned field was dropped, as this is future information and it would be contaminating our training dataset.
- All occurrences for Tenure = 0, were dropped. This is an interim situation, where Avg_Monthly_Charge is also “0”, but the rest of the features defining the services or payment types were not “0” or null. These rows accounted for new customers that have not paid any fees yet. So, there would be nothing to predict for them. The zero Avg_Monthly_Charge would only contaminate with noise to the dataset. This was dropped though after we plotted the first scatterplots of y_pred vs. y_test (or the actuals) and these points were identified to be different.
- The dataset was split into train-test datasets in a 70 / 30 proportion

One Hot Encoder Transformation with ColumnTransformer

As mentioned in the analysis “Multiple Categorical Features” in EDA, the “.get_dummies” was applied to the whole dataset for encoding the categorical variables.

In this part of the analysis, we applied the encoding of categorical features after the train/ test split, to prevent leakage of information to the test data and to see any impact on the analysis already made in the preceding chapters.

In a production environment, direct application of “.get_dummies” after a train-test split can be problematic as the test set might not have the same exact produced features. This will lead to potential errors in downstream ML models. To prevent such issues, the **ColumnTransformer** technique was adopted with the use of **OneHotEncoding** transformation for categorical features. The issue of difference in the produced features is handled behind the scenes through

appropriate parameters (`handle_unknown = 'ignore'`). Also, the first of the produced columns was dropped (`drop = 'first'`), to mitigate collinearity, which is crucial for regression models.

Numerical features (Tenure) were transformed through **StandardScaler** ensuring values will be comparable to the rest of the features that will have distinct values of 0 and 1 through OHE.

Finally, we renamed all produced features to comply with the naming in EDA.

Correlation between variables after OHE

After checking the correlation between variables of the training set, very strong correlations were identified

- between fields that were created with the *<special Service>_NoInternet* suffix and the field *Is_InternetService_None*.

Hence, instead of having 6 perfectly correlated columns indicating no special service due to Internet service not being available in the contract, there will be only one field indicating no internet service. The same approach was followed for the *_NoPhone* suffix of services. The following features were dropped:

'Has_OnlineSecurity_NoInternet'
'Has_OnlineBackup_NoInternet'
'Has_DeviceProtection_NoInternet'
'Has_TechSupport_NoInternet'
'Has_StreamingTV_NoInternet',
'Has_StreamingMovies_NoInternet'
'Has_StreamingMovies_NoInternet'
'Has_MultipleLines_NoPhone',

Table 11 *Dropped highly correlated fields*

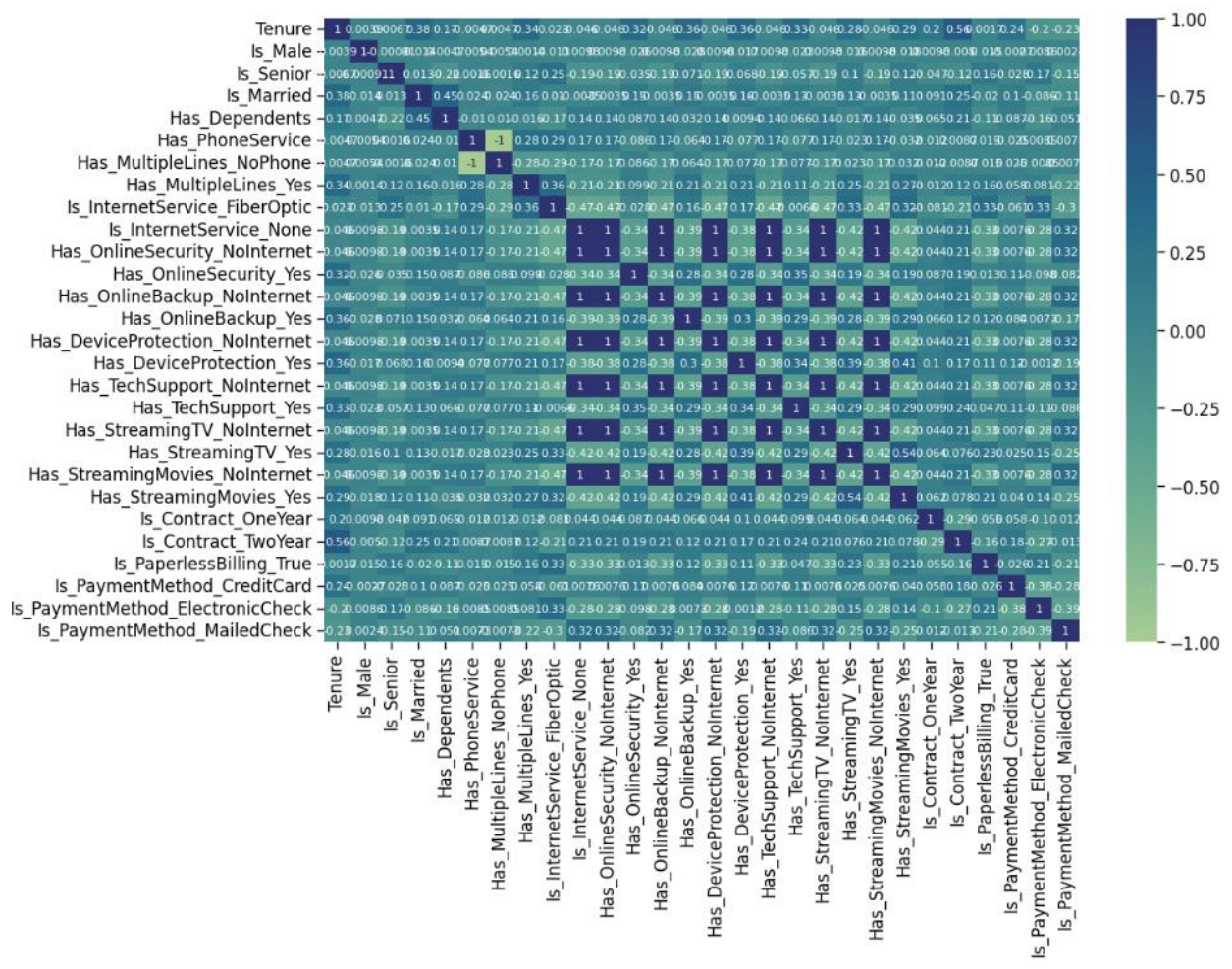


Figure 7 Correlation heatmap demonstrating that the “NoInternet” labeled fields are highly correlated with the Is_InternetService_None

The remaining dataset correlations may be found below:

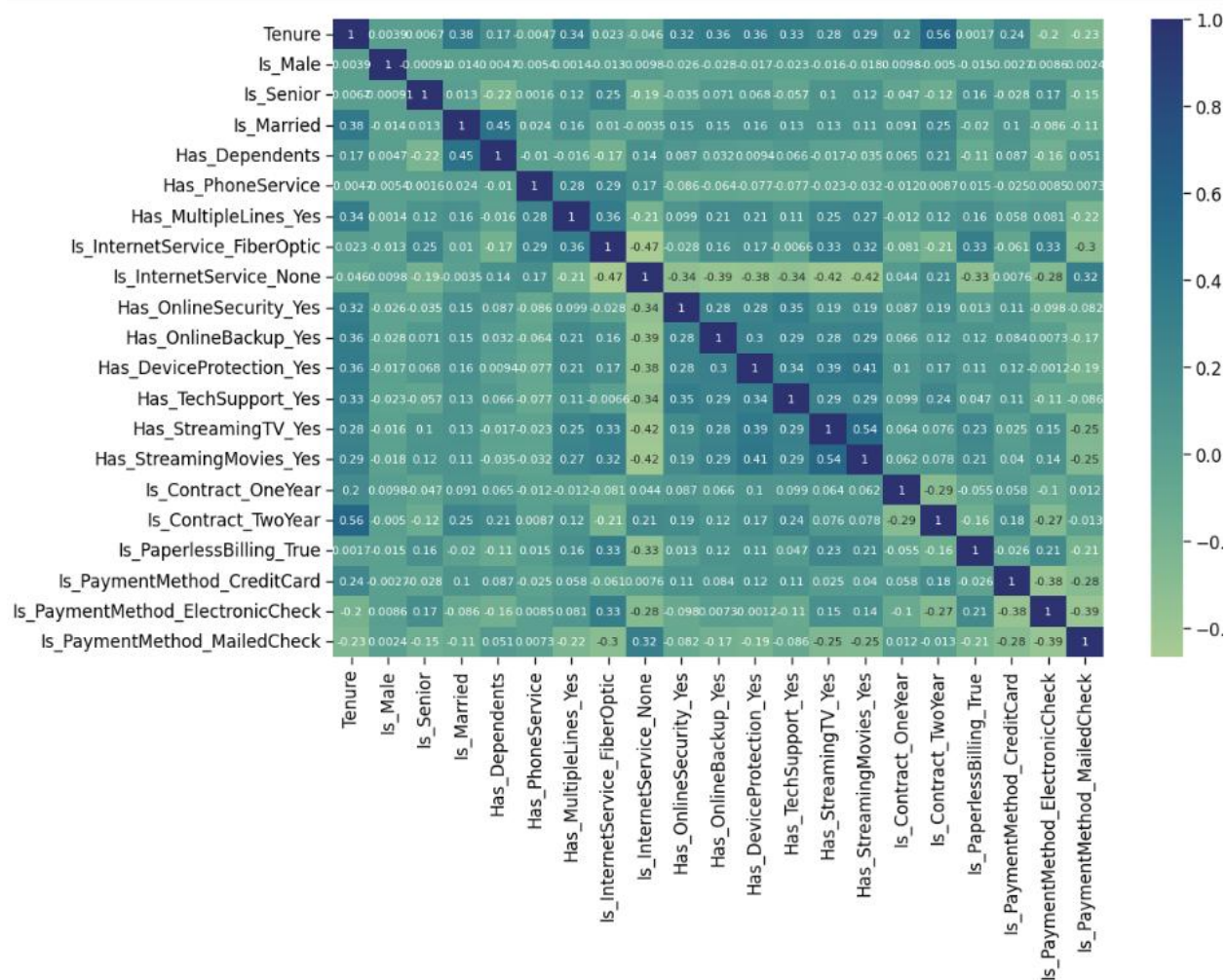


Figure 8 Correlation heatmap of Train dataset where highly correlated fields have been removed.

Baseline Regression Models.

With the reduced set of features with no strong correlations amongst them, the following prediction algorithms were run, as shown in the table below. The second part of the table shows the same models where Tenure=0 – rows were excluded from the dataset.

Model	R^2	MAE	Parameters
XGBRegressor	0.986	2.207	
Ridge	0.985	2.090	

Linear Regression	0.985	2.090	
Lasso	0.985	2.085	
SVR Linear	0.985	2.075	
RandomForest	0.983	2.411	
KNN	0.955	4.289	
Tenure > 0			
SVR Linear	0.991	2.020	
Linear Regression	0.991	2.018	
Lasso	0.991	2.018	0.01
Ridge	0.991	2.018	0.1
XGBRegressor	0.990	2.122	
RandomForest	0.988	2.337	
SVR NonLinear	0.987	2.418	
KNN	0.956	4.354	

Table 12 Baseline models and their metrics R^2 and MAE. Categorized with descending R^2 . We see R^2 increasing and MAE dropping with the influence of the Tenure = 0 removal.

By dropping 11 out of ~7000 rows with Tenure = 0, the metrics improve. This shows how regressor algorithms depend on small details and outliers.

This analysis shows that Linear Regression models work very well for this dataset for predicting the monthly spend of a customer. The predicted variable is a linear combination of the most important features identified.

Feature Engineering

The effect of how family conditions might affect monthly spending was modelled through the introduction of an engineered feature, which was called “Household_Type”. This was derived from ‘Is_Married’ and ‘Has_Dependents’, following the logic below:

Is_Married	Has_Dependents	Household_Type
True	True	Family
True	False	Couple

False	True	Single Parent
False	False	Single Adult

Table 13 New Engineered Feature as a combination of someone being married and having dependents

For evaluating the effect of this additional feature, the best performing models from the previous analysis were utilized (Table 12). Specifically, the models deployed were Lasso - as a linear regression model capable of feature elimination and XGBRegressor - as a more complex model relying on decision trees- to capture possible non-linear behaviors.

Additionally, as a follow up step, since both Lasso and XGBRegressor identified features with zero coefficients, these were eliminated from the X_train and X_test datasets, to see the impact on model performance.

Finally, XGBRegressor was used in a Randomized Search CV Grid to identify the most optimal parameters for the model to predict monthly spend.

The exact same steps were followed as with the previous analysis. The final performance metrics may be seen below:

Model	R ²	MAE	Parameter
Lasso & Household Features	0.991	2.019	0.01
XGBRegressor & Household Features	0.990	2.124	--
Lasso & Selected Important Features	0.991	2.018	0.001
XGBRegressor & Selected Important Features	0.990	2.120	--
XGBRegressor Optimized with Grid Search	0.991	2.022	

Table 14 Metrics for Lasso and XGBRegressor for additional household features, with only selected features having a weight >0 from Lasso regression and final XGBRegressor optimized with CV Grid. As the features are more "selected", the alpha parameter becomes smaller, indicating more dominating weights.

- The family conditions field did not have any predictive importance for the average monthly spend.
- Eliminating Lasso identified zero-coefficient features; small improvements in performance were identified.

Model Interpretation

Below we may find the most important features that govern the average monthly spend, seen by Lasso and the XGBRegressor.

SHAP, apart from Coefficients (for Lasso) and Feature Importances (for XGBRegressor) was used to explain exactly what drives a customer's bill up or down. We find

- **Top Drivers:** According to both models, **Fiber Optic** and **Phone Service** are the massive price drivers. Additional **Streaming Services** play a vital role.
- **Adverse Impact:** If there are **no Internet Services** then the spend is significantly smaller
- **Zero Impact:** Engineered features like Household_Type (Family vs. Single) have **zero** predictive importance—spending is driven by service choice, not family status.
- **Conclusion:** We achieved a near-perfect prediction ($R^2 = 0.991$) with a Mean Absolute Error of only **\$2.02**. The model is highly deployable and transparent.

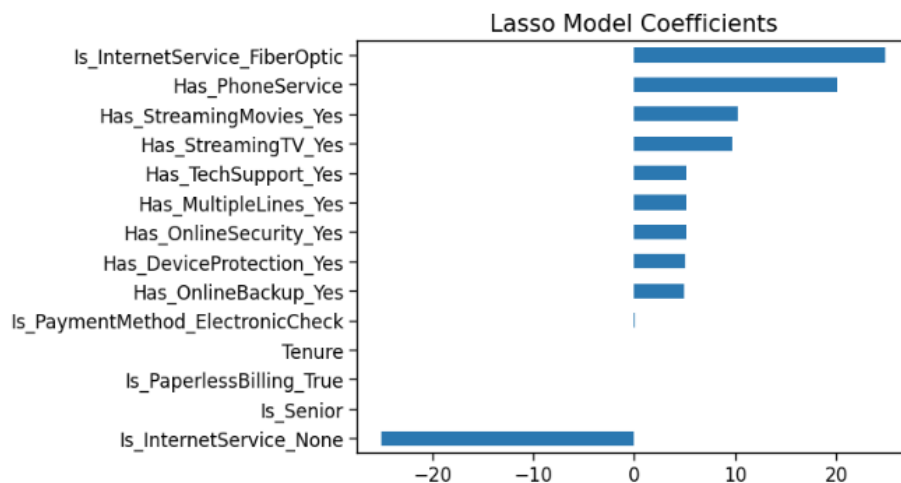


Figure 9. Lasso Coefficients. Magnitude and Direction is shown as coefficients can be positive or negative

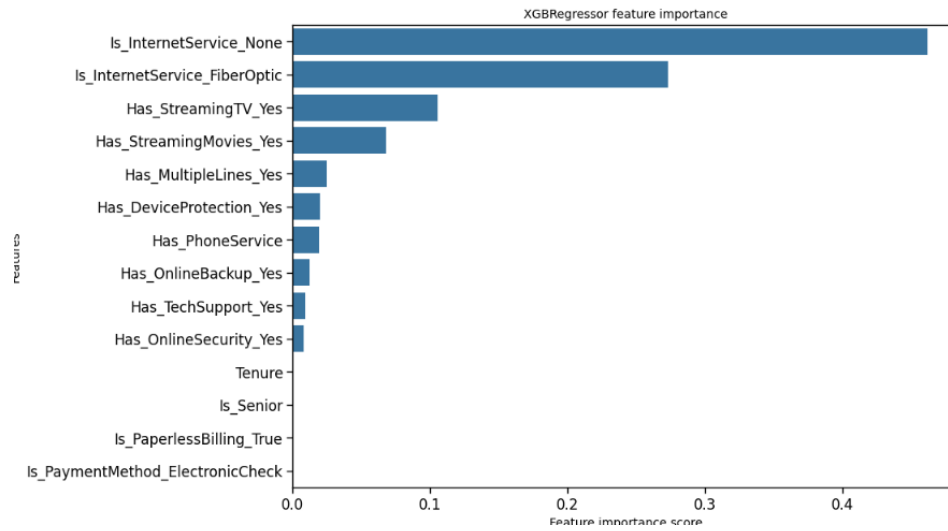


Figure 10 XGBRegressor: Tree based Feature Importances. Direction is not captured through importances, only magnitude

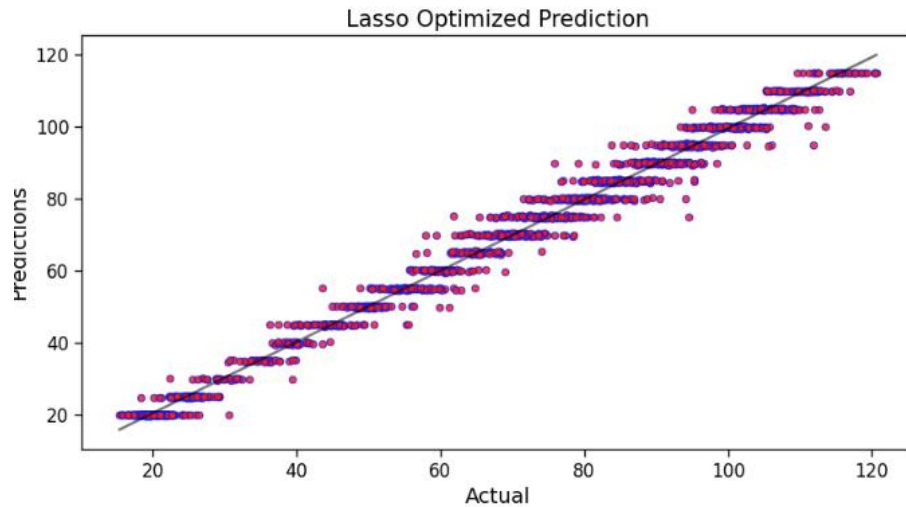


Figure 11 Lasso predictions vs. Actuals fall extremely near to XGBRegressor as seen in <>

The relationship of monthly spending with the most important features kept, is linear. Both models (Figure 11 & Figure 12) capture **the 'horizontal strips' which might be the actual values that have a dispersion due to Tenure**, while the jump in the predicted values shows that the model captures the itemization according to the **fixed pricing plans of the telco company**. It reveals the actual pricing formula.

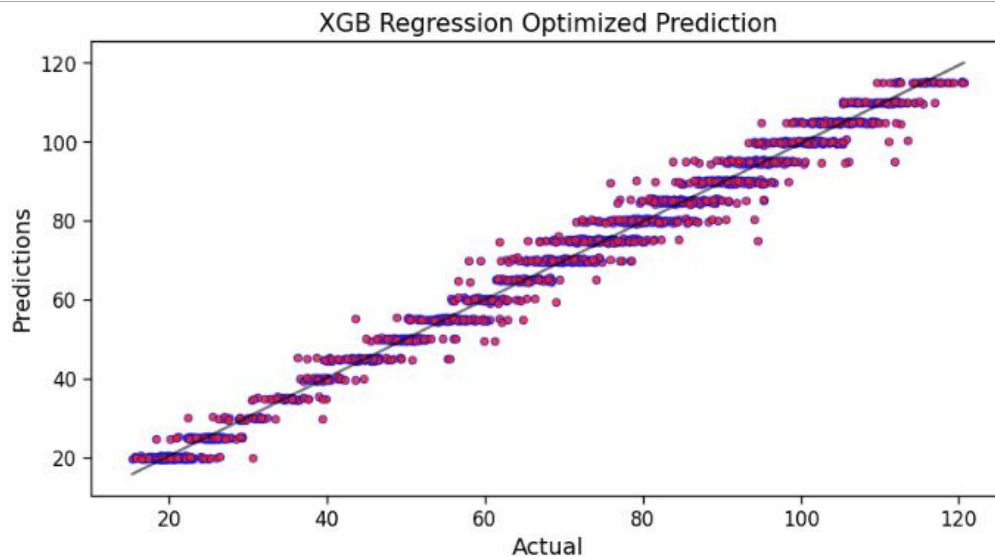


Figure 12 XGBRegressor showing the “same” results as Lasso Figure 11. Through CV Grid Best parameters it “downgrades” itself to a simpler shallower model with `max_depth = 1`.



Figure 13 : Shap LinearExplainer for Lasso. To also see the direction of the impact the beeswarm plot is needed and may be found in the regression Python notebook

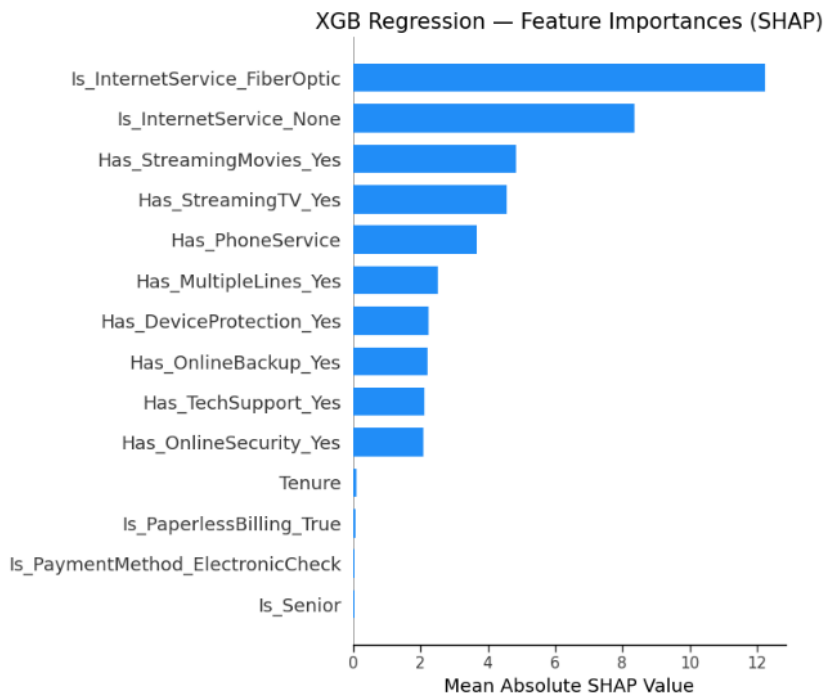


Figure 14 Shap Tree Explainer for tree-based models like XGBRegressor. To also see the direction of the impact the beeswarm plot is needed and may be found in the regression Python notebook

Most important parameters affecting customer spend

#1 <u>Is_InternetService_FiberOptic</u>	+ ~€12/mo	#2 <u>Has_StreamingMovies / Has_StreamingTV</u>	+€5-8/mo
#3 <u>Has_PhoneService / MultipleLines</u>	+€3-5/mo	#4 <u>Is_InternetService_None</u>	-€20/mo

Figure 15. Most important factors affecting customer spend and their \$ impact on the monthly spend through MAE

Additional Model Approach

Instead of a 70/30 train-test split, a test was performed with an 80/20 split to observe results and compare with the rest of the tests done so far. The dataset was trained for **Classification** with XGBoost and Logistic Regression and for **Regression** with XGBoost Regressor and Ridge Regression. This part of the project can be considered an independent approach.

Churn Classification

Fields used for the Classification are described in the EDA (Figure 3) , after .get_dummies, and also without “Total_Charges”, since Avg_Monthly_Charges was derived directly by it.

The split between churned vs. not churned in the dataset was 1,869 / 5,174 (or 73/27 in percentages). There was an attempt to utilize SMOTE to balance the dataset. The models performed worse on the test set, though, reducing the F1 score from 0.6289 to 0.6197, indicating that the model was overfitting the synthetic rows rather than learning better patterns. Hence, it was decided to handle the imbalance directly inside the models using ‘scale_pos_weight’ for XGBoost and ‘class_weight’ for Logistic Regression.

Both models for churn prediction were tuned using GridSearchCV with 5-fold cross-validation, optimizing for F1 score, since accuracy would tend to predict the majority class with 73%.

Tuning & Best Parameters

XGBoost best parameters : learning_rate = 0,05, max_depth= 3, n_estimators = 200

Logistic Regression best parameters : C=0.1, Solver = liblinear

XGBoost best performance was achieved with shallow trees (max_depth), preventing “noise” memorization. For Logistic Regression the liblinear solver created a regularized boundary, which was expected for a dataset which with no complexity.

Results

The results on the test set were the following:

Metric	XGBoost	Logistic Regression	Winner
Accuracy	0.7488	0.7431	XGBoost
F1 Score	0.6289	0.6173	XGBoost
Recall (Churners)	0.80	0.78	XGBoost
Precision (Churners)	0.52	0.51	Tied

Table 15 Model Performance Comparison: XGBoost vs Logistic Regression

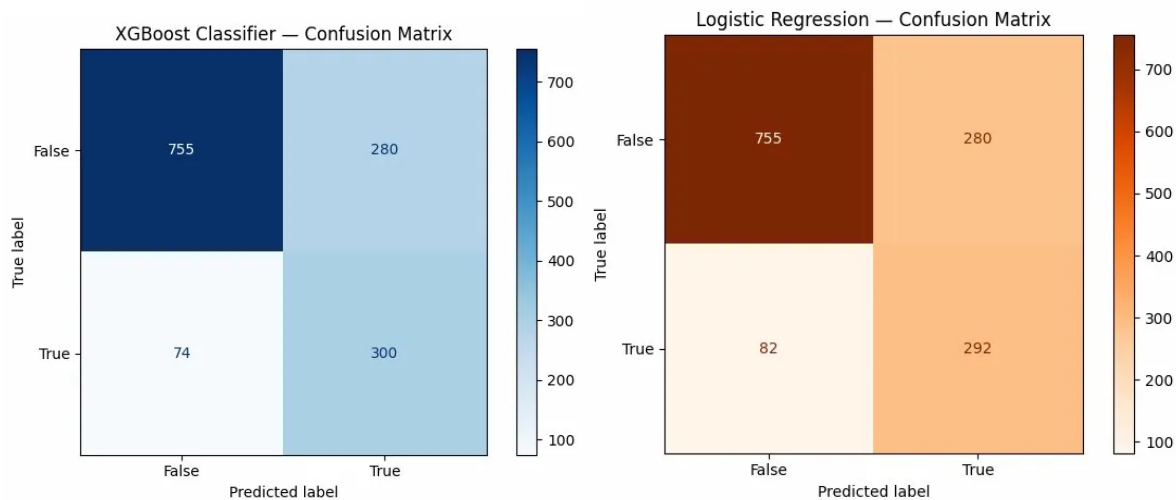


Figure 16 Confusion Matrix Comparison: XGBoost vs Logistic Regression

We focused on **Recall** (True Positives / Actual Positives) because a missed churner (False Negative) has significantly more 'business cost' than of a false positive, which would result in an unnecessary retention offer (False Positive).

Model	Recall	Churners Caught	Churners Missed
XGBoost	0.80	300 / 374	74
Logistic Regression	0.78	292 / 374	82

Table 16 Classification model performances and explanation

- XGBoost achieved the highest recall, capturing 80% of potential churners.
- Logistic Regression although a more simplistic model, performed very well, falling back on XGBoost with only 8 customers.
- The similarity of models suggests that churn for this dataset is linear and no highly complex modeling is needed.
- By adopting XGBoost, the business successfully identifies 300 at-risk customers, allowing for proactive intervention where the cost of a "false alarm" (a redundant discount) is negligible compared to total customer loss.

What Is Driving Churn? (SHAP)

Rank	Feature	XGBoost SHAP	Logistic Reg SHAP
1	Tenure	0.62	0.72
2	Is_Contract_TwoYear	0.53	0.39
3	Avg_Monthly_Charge / FiberOptic	0.28	0.34

Table 17 Top Feature Importance Comparison Using SHAP (XGBoost vs Logistic Regression)

Through SHAP the primary features affecting churn were identified, being the same for both models. **Tenure** is the strongest predictor; meaning that churn risk decreases significantly as customer longevity increases. **The Contract Type**, high commitment (two-year) contracts signify lower churn compared to month-to-month plans. And finally, **the Service Tier/Cost**. Fiber Optic subscriptions contribute to larger churn. (Table 17)

The conclusion is that higher-paying customers on premium services are much more likely to “shop around”

Monthly Charge Regression Analysis

The ‘Is_Churned’ column was also dropped, as this would be data not available yet (future conclusion), in a real business environment.

Regression models testing aimed at predicting the customer's average monthly charge. The chosen models were tuned with GridSearchCV using R^2 as the scoring metric.

Tuning & Best Parameters

XGBoost Regressor best parameters : learning_rate = 0.1, max_depth= 3, n_estimators = 200

Ridge Regression best parameters : alpha = 0.01

Alpha parameter suggests that there is very little regularization, which comes to contradicting the Lasso Regression tested in [Regression Analysis](#). There we had deleted the highly correlated fields.

Results

Metric	XGBoost	Ridge Regression	Winner
R^2 Score	0.9905	0.9871	XGBoost
MAE	2.09	2.04	Ridge
Train R^2	0.9897	0.9877	XGBoost

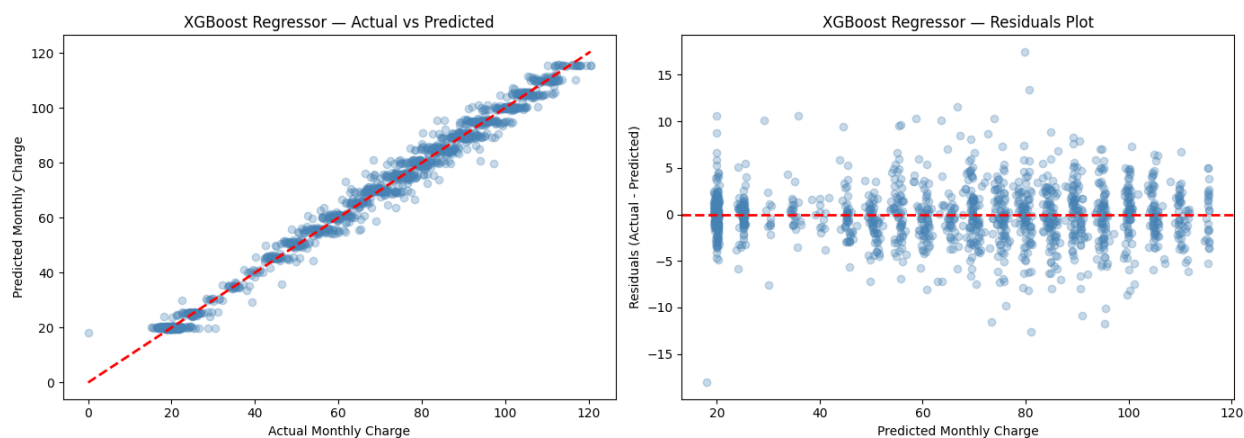
Table 18 Regression Model Performance Comparison: XGBoost vs Ridge Regression

Both models performed well. XGBoost hit an R^2 of 0.9905 on the test set, meaning it explains 99% of the variance in monthly charges, with an average error of just 2.09 \$/month on a scale of 0 to 121. Ridge was right behind it with an R^2 of 0.9871 and edged ahead on MAE at 2.04 \$/month.

XGBoost Regressor and Ridge have shown to behave similarly. This indicates that monthly charges in this dataset are a direct sum of whichever services a customer subscribes to, which is the norm for a telecom company where bills are itemized per service and is in sink with the analysis performed in chapter [Regression Analysis](#).

Actual vs Predicted and Residuals

The Actual vs Predicted plots for both models reveal that the data points are sitting perfectly along the diagonal line. The residuals (the difference between actuals and predicted) are scattered randomly around zero for both models, mostly within the -5 to +5 range, with no pattern prevailing. This suggests that the models are not consistently over or under-predicting in any region.



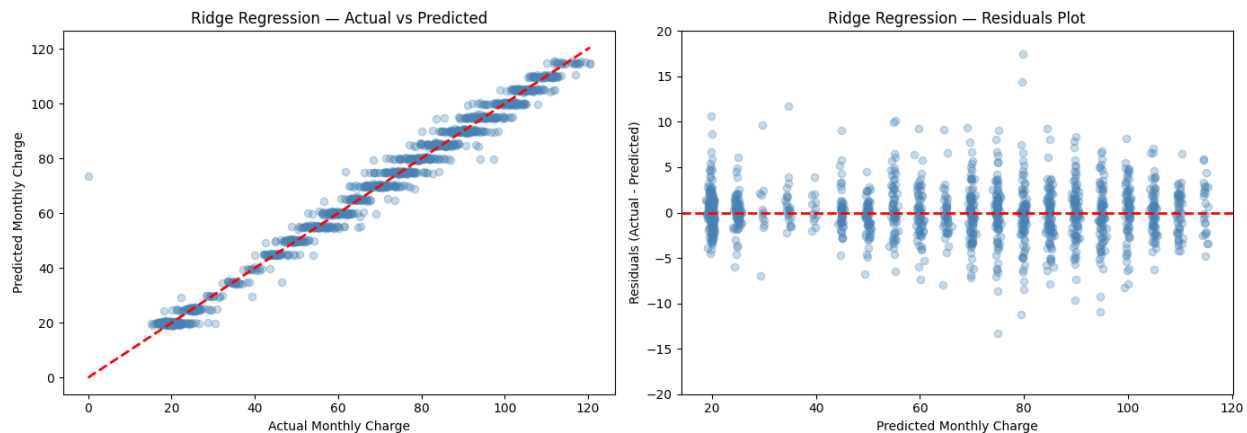


Figure 17 Regression Model Diagnostics: Actual vs Predicted and Residual Analysis

What Drives Monthly Charges? (SHAP)

Rank	Feature	XGBoost SHAP	Ridge SHAP
1	Is_InternetService_FiberOptic	16.7	11.9
2	Has_OnlineSecurity_NoInternet/Has_StreamingMovies_No	15.1	4.8
3	Has_StreamingMovies_No/Has_StreamingTV_No	4.0	4.6

Table 19 Top Feature Importance Comparison (Regression) Using SHAP: XGBoost vs Ridge

The results above coincide with Figure 15, coming from the [Regression Analysis](#) chapter.

Despite the differences in importances of secondary features, for both models we reach the same conclusion: what a customer pays each month comes down to their internet service type and which add-ons they subscribe to on top of it.

Conclusion

To conclude, this project used supervised machine learning to address two business problems in telecommunications, specifically predicting customer churn and estimating average monthly charges. For predicting the churners, the best model was the tuned SVM with an RBF kernel, successfully identifying 75% of churners, which significantly overachieved over the Random Forest's 48% recall. For regression, both linear and tree-based boosting models achieved almost perfect performance. The boosting regression algorithm downgraded itself to a very shallow tree, confirming that monthly charges are a linear combination of subscribed services, hence described by a highly deployable and transparent model. These findings were also confirmed by the approach of 80/20 split with XGBoost, Logistic Regression, and Ridge Regression.

The models delivered valuable results and understanding of the dynamics of customer churn and spend. These findings can be used by the company, enabling efficient retention targeting and supporting data-driven pricing strategy. Together, they provide a practical way to reduce customer attrition and improve revenue.